From Ground Up: Programming Problems + Conceptual Questions

For each keyword, complete one programming challenge and answer three conceptual questions. Use basic Python first where practical; verify with tests and hand calculations.

Phase 0 — Computational thinking and basic programming

Algorithms

Programming problem: Implement a function that finds the second-largest distinct number in a list without sorting it. Write down the algorithm before coding.

Questions you should be able to answer:

1. What makes an algorithm correct, not merely able to run?
2. How would you estimate its time and memory costs as input size grows?
3. Which input is the best counterexample to a flawed solution?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Problem decomposition

Programming problem: Take “analyze a text file” and split it into small functions with clear inputs and outputs; implement the smallest working version.

Questions you should be able to answer:

1. How do you turn a vague goal into smaller testable tasks?
2. Which subtask has a clear interface and can be tested independently?
3. What dependency between tasks determines the order of implementation?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Inputs and outputs

Programming problem: Write a function that receives a list of numbers and returns count, minimum, maximum, and average; validate empty input.

Questions you should be able to answer:

1. Explain inputs and outputs in your own words and give a minimal example.
2. What changes during execution, and how can you trace that change?
3. What common mistake or edge case would cause a wrong result, and how would you test for it?

☐ Implemented - [ ] Tested - [ ] Explained from memory

State

Programming problem: Simulate a bank account with a balance variable and deposit/withdraw operations; print the state after every operation.

Questions you should be able to answer:

1. Explain state in your own words and give a minimal example.
2. What changes during execution, and how can you trace that change?
3. What common mistake or edge case would cause a wrong result, and how would you test for it?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Variables

Programming problem: Write a small standalone Python exercise demonstrating variables. Print or inspect intermediate state, assert the expected result, and include one boundary or failure case. Avoid a library that performs the main task for you.

Questions you should be able to answer:

1. Explain variables in your own words and give a minimal example.
2. What changes during execution, and how can you trace that change?
3. What common mistake or edge case would cause a wrong result, and how would you test for it?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Python functions

Programming problem: Write a reusable function with clear parameters and a return value; test repeated calls, invalid input, and whether it has side effects.

Questions you should be able to answer:

1. What is the difference between a parameter, an argument, and a return value?
2. Why can a pure function be easier to test than one with side effects?
3. What should your function do for invalid or boundary input?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Expressions

Programming problem: Write a small standalone Python exercise demonstrating expressions. Print or inspect intermediate state, assert the expected result, and include one boundary or failure case. Avoid a library that performs the main task for you.

Questions you should be able to answer:

1. Explain expressions in your own words and give a minimal example.
2. What changes during execution, and how can you trace that change?
3. What common mistake or edge case would cause a wrong result, and how would you test for it?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Boolean logic

Programming problem: Write an eligibility checker with at least three conditions; express it first as a truth table, then as Python logic.

Questions you should be able to answer:

1. Explain boolean logic in your own words and give a minimal example.
2. What changes during execution, and how can you trace that change?
3. What common mistake or edge case would cause a wrong result, and how would you test for it?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Conditionals

Programming problem: Classify a numeric score into named bands; test exact boundary values and invalid input.

Questions you should be able to answer:

1. Explain conditionals in your own words and give a minimal example.
2. What changes during execution, and how can you trace that change?
3. What common mistake or edge case would cause a wrong result, and how would you test for it?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Loops

Programming problem: Compute the running sum and running average of numbers without using sum() or statistics.mean().

Questions you should be able to answer:

1. Explain loops in your own words and give a minimal example.
2. What changes during execution, and how can you trace that change?
3. What common mistake or edge case would cause a wrong result, and how would you test for it?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Recursion

Programming problem: Implement factorial and recursively sum a nested list; identify and test the base case.

Questions you should be able to answer:

1. Explain recursion in your own words and give a minimal example.
2. What changes during execution, and how can you trace that change?
3. What common mistake or edge case would cause a wrong result, and how would you test for it?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Tracing

Programming problem: Trace a nested-loop program by hand, then add print statements to confirm each variable value.

Questions you should be able to answer:

1. Explain tracing in your own words and give a minimal example.
2. What changes during execution, and how can you trace that change?
3. What common mistake or edge case would cause a wrong result, and how would you test for it?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Time complexity

Programming problem: Count operations for linear search, binary search, and a nested-loop pair counter for increasing input sizes; compare growth.

Questions you should be able to answer:

1. What input size n does the complexity describe?
2. How do O(1), O(log n), O(n), and $O(n^2)$ differ as n grows?
3. How would you distinguish measured runtime from asymptotic complexity?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Debugging

Programming problem: Fix a deliberately buggy function with at least three faults; reproduce each fault with a test before changing code.

Questions you should be able to answer:

1. Explain debugging in your own words and give a minimal example.
2. What changes during execution, and how can you trace that change?
3. What common mistake or edge case would cause a wrong result, and how would you test for it?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Phase 1 — Python deeply and object-oriented programming

Mutable objects

Programming problem: Show how mutating a list through one variable affects another variable that refers to the same list; then avoid the aliasing bug.

Questions you should be able to answer:

1. What is the difference between mutating an object and rebinding a variable?
2. How can aliasing cause an unexpected result?
3. When is a shallow copy insufficient?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Immutable objects

Programming problem: Demonstrate strings and tuples cannot be mutated in place; write a function that returns a modified replacement instead.

Questions you should be able to answer:

1. Explain immutable objects in your own words and give a minimal example.
2. What changes during execution, and how can you trace that change?
3. What common mistake or edge case would cause a wrong result, and how would you test for it?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Object references

Programming problem: Create two references to one object and compare aliasing with a shallow copy and a deep copy using nested lists.

Questions you should be able to answer:

1. What does a variable hold when it refers to a list?
2. How can two variables refer to the same object?
3. How can you test identity versus equality?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Scope

Programming problem: Write functions with local and global variables, then explain which binding each expression reads.

Questions you should be able to answer:

1. Explain scope in your own words and give a minimal example.
2. What changes during execution, and how can you trace that change?
3. What common mistake or edge case would cause a wrong result, and how would you test for it?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Closures

Programming problem: Create a function that returns a counter function; keep the counter state without using a global variable.

Questions you should be able to answer:

1. What values does an inner function retain from its enclosing scope?
2. How does a closure differ from a global variable?
3. What behavior would a late-binding loop closure produce?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Iterators

Programming problem: Write a custom iterator that returns the first n squares and raises StopIteration at the correct time.

Questions you should be able to answer:

1. Explain iterators in your own words and give a minimal example.
2. What changes during execution, and how can you trace that change?
3. What common mistake or edge case would cause a wrong result, and how would you test for it?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Generators

Programming problem: Rewrite the square iterator as a generator; compare its values and memory behavior with a list.

Questions you should be able to answer:

1. Explain generators in your own words and give a minimal example.
2. What changes during execution, and how can you trace that change?
3. What common mistake or edge case would cause a wrong result, and how would you test for it?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Comprehensions

Programming problem: Convert a loop that filters and transforms numbers into a list comprehension; add a condition and compare both outputs.

Questions you should be able to answer:

1. Explain comprehensions in your own words and give a minimal example.
2. What changes during execution, and how can you trace that change?
3. What common mistake or edge case would cause a wrong result, and how would you test for it?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Exceptions

Programming problem: Write a safe integer parser that catches only the expected errors and reports invalid values clearly.

Questions you should be able to answer:

1. Explain exceptions in your own words and give a minimal example.
2. What changes during execution, and how can you trace that change?
3. What common mistake or edge case would cause a wrong result, and how would you test for it?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Context managers

Programming problem: Write a context manager that opens a file and guarantees cleanup, including when an exception occurs.

Questions you should be able to answer:

1. Explain context managers in your own words and give a minimal example.
2. What changes during execution, and how can you trace that change?
3. What common mistake or edge case would cause a wrong result, and how would you test for it?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Modules

Programming problem: Split a script into a reusable module and a main program; prevent test code from running on import.

Questions you should be able to answer:

1. Explain modules in your own words and give a minimal example.
2. What changes during execution, and how can you trace that change?
3. What common mistake or edge case would cause a wrong result, and how would you test for it?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Packages

Programming problem: Create a small package with two modules and imports; run it from the project root and test import paths.

Questions you should be able to answer:

1. Explain packages in your own words and give a minimal example.

2. What changes during execution, and how can you trace that change?
3. What common mistake or edge case would cause a wrong result, and how would you test for it?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Type hints

Programming problem: Add type hints to a function that consumes records and returns summary statistics; test valid and invalid shapes.

Questions you should be able to answer:

1. Explain type hints in your own words and give a minimal example.
2. What changes during execution, and how can you trace that change?
3. What common mistake or edge case would cause a wrong result, and how would you test for it?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Classes

Programming problem: Implement a Student class with state and methods; create several instances with independent data.

Questions you should be able to answer:

1. What behavior and state belong together in a class?
2. When is a plain function or dataclass simpler than a custom class?
3. How would you test two instances for accidental shared state?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Instances

Programming problem: Create two instances of the same class, mutate one, and demonstrate which values remain independent.

Questions you should be able to answer:

1. Explain instances in your own words and give a minimal example.
2. What changes during execution, and how can you trace that change?
3. What common mistake or edge case would cause a wrong result, and how would you test for it?

☐ Implemented - [ ] Tested - [ ] Explained from memory

self

Programming problem: Write a method that updates an instance field; explain what object self refers to when called.

Questions you should be able to answer:

1. Explain self in your own words and give a minimal example.
2. What changes during execution, and how can you trace that change?
3. What common mistake or edge case would cause a wrong result, and how would you test for it?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Constructors

Programming problem: Validate constructor arguments and establish class invariants when an object is created.

Questions you should be able to answer:

1. Explain constructors in your own words and give a minimal example.
2. What changes during execution, and how can you trace that change?
3. What common mistake or edge case would cause a wrong result, and how would you test for it?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Composition

Programming problem: Build a Trainer that contains a Model and an Optimizer object instead of inheriting their behavior.

Questions you should be able to answer:

1. Why might an object contain another object rather than inherit from it?
2. How does composition make components replaceable?
3. What happens if the contained object changes interface?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Inheritance

Programming problem: Create a base Shape class and two subclasses; identify which common behavior belongs in the base class.

Questions you should be able to answer:

1. What relationship should justify inheritance?
2. What is the risk of overriding a method with incompatible behavior?
3. When would composition be more appropriate?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Polymorphism

Programming problem: Give two model classes the same predict() interface and run them through one evaluation function.

Questions you should be able to answer:

1. How can unrelated classes be used through the same interface?
2. What contract must every implementation honor?
3. How does polymorphism reduce conditional logic in a caller?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Abstract base classes

Programming problem: Define an abstract Model interface that requires predict(); show that incomplete subclasses cannot be instantiated.

Questions you should be able to answer:

1. Explain abstract base classes in your own words and give a minimal example.
2. What changes during execution, and how can you trace that change?
3. What common mistake or edge case would cause a wrong result, and how would you test for it?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Protocols

Programming problem: Define a typing Protocol for objects with predict(); use it without requiring a shared parent class.

Questions you should be able to answer:

1. Explain protocols in your own words and give a minimal example.
2. What changes during execution, and how can you trace that change?
3. What common mistake or edge case would cause a wrong result, and how would you test for it?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Dataclasses

Programming problem: Represent an experiment configuration with a dataclass, defaults, and validation for invalid parameter values.

Questions you should be able to answer:

1. Explain dataclasses in your own words and give a minimal example.

2. What changes during execution, and how can you trace that change?
3. What common mistake or edge case would cause a wrong result, and how would you test for it?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Unit testing

Programming problem: Test a pure function independently using expected results and boundary cases.

Questions you should be able to answer:

1. Explain unit testing in your own words and give a minimal example.
2. What changes during execution, and how can you trace that change?
3. What common mistake or edge case would cause a wrong result, and how would you test for it?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Phase 2A — Algebra and functions

Variables in algebra

Programming problem: Write an evaluator for an expression such as 3*x + 2 and test several values of x.

Questions you should be able to answer:

1. State the definition of variables in algebra and explain its intuitive or geometric meaning.
2. Work through one small numerical example by hand. What are the dimensions and units, if applicable?
3. What assumptions or conditions are required, and how would you verify your implementation?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Equations

Programming problem: Write a solver for a linear equation ax+b=0 and handle a=0 separately.

Questions you should be able to answer:

1. State the definition of equations and explain its intuitive or geometric meaning.
2. Work through one small numerical example by hand. What are the dimensions and units, if applicable?
3. What assumptions or conditions are required, and how would you verify your implementation?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Inequalities

Programming problem: Write a function that classifies x relative to a threshold and test <, <=, >, and >= boundary cases.

Questions you should be able to answer:

1. State the definition of inequalities and explain its intuitive or geometric meaning.
2. Work through one small numerical example by hand. What are the dimensions and units, if applicable?
3. What assumptions or conditions are required, and how would you verify your implementation?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Powers

Programming problem: Implement integer exponentiation by repeated multiplication, then compare with the built-in ** operator.

Questions you should be able to answer:

1. State the definition of powers and explain its intuitive or geometric meaning.
2. Work through one small numerical example by hand. What are the dimensions and units, if applicable?
3. What assumptions or conditions are required, and how would you verify your implementation?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Logarithms

Programming problem: Implement log base b by solving b**x≈n with binary search for positive inputs; compare with math.log.

Questions you should be able to answer:

1. State the definition of logarithms and explain its intuitive or geometric meaning.
2. Work through one small numerical example by hand. What are the dimensions and units, if applicable?
3. What assumptions or conditions are required, and how would you verify your implementation?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Functions

Programming problem: Implement a pure function and an intentionally side-effecting function; compare repeatability for identical inputs.

Questions you should be able to answer:

1. State the definition of functions and explain its intuitive or geometric meaning.
2. Work through one small numerical example by hand. What are the dimensions and units, if applicable?
3. What assumptions or conditions are required, and how would you verify your implementation?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Function composition

Programming problem: Implement f(x)=2x+1 and g(x)=xx, then calculate f(g(x)) and g(f(x)) to show order matters.

Questions you should be able to answer:

1. State the definition of function composition and explain its intuitive or geometric meaning.
2. Work through one small numerical example by hand. What are the dimensions and units, if applicable?
3. What assumptions or conditions are required, and how would you verify your implementation?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Inverse functions

Programming problem: Implement a simple invertible function and its inverse; test that composing them approximately recovers the input.

Questions you should be able to answer:

1. State the definition of inverse functions and explain its intuitive or geometric meaning.
2. Work through one small numerical example by hand. What are the dimensions and units, if applicable?
3. What assumptions or conditions are required, and how would you verify your implementation?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Coordinate systems

Programming problem: Convert points between Cartesian and polar coordinates and test the origin and quadrant boundaries.

Questions you should be able to answer:

1. State the definition of coordinate systems and explain its intuitive or geometric meaning.
2. Work through one small numerical example by hand. What are the dimensions and units, if applicable?
3. What assumptions or conditions are required, and how would you verify your implementation?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Lines

Programming problem: Given two points, calculate the line equation and predict y for a new x; handle a vertical line.

Questions you should be able to answer:

1. State the definition of lines and explain its intuitive or geometric meaning.

2. Work through one small numerical example by hand. What are the dimensions and units, if applicable?
3. What assumptions or conditions are required, and how would you verify your implementation?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Slopes

Programming problem: Calculate slope from pairs of points and estimate local slope from nearby samples of a curve.

Questions you should be able to answer:

1. State the definition of slopes and explain its intuitive or geometric meaning.
2. Work through one small numerical example by hand. What are the dimensions and units, if applicable?
3. What assumptions or conditions are required, and how would you verify your implementation?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Polynomials

Programming problem: Represent a polynomial as a coefficient list and evaluate it using loops or Horner's method.

Questions you should be able to answer:

1. State the definition of polynomials and explain its intuitive or geometric meaning.
2. Work through one small numerical example by hand. What are the dimensions and units, if applicable?
3. What assumptions or conditions are required, and how would you verify your implementation?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Exponential functions

Programming problem: Tabulate and plot b**x for several bases; demonstrate how growth changes with the base.

Questions you should be able to answer:

1. State the definition of exponential functions and explain its intuitive or geometric meaning.
2. Work through one small numerical example by hand. What are the dimensions and units, if applicable?
3. What assumptions or conditions are required, and how would you verify your implementation?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Phase 2B — Linear algebra

Scalars

Programming problem: Write arithmetic helpers for scalar inputs and reject incompatible types with a clear error.

Questions you should be able to answer:

1. State the definition of scalars and explain its intuitive or geometric meaning.
2. Work through one small numerical example by hand. What are the dimensions and units, if applicable?
3. What assumptions or conditions are required, and how would you verify your implementation?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Vectors

Programming problem: Represent vectors as lists and implement length checking, addition, and a readable vector printer.

Questions you should be able to answer:

1. State the definition of vectors and explain its intuitive or geometric meaning.
2. Work through one small numerical example by hand. What are the dimensions and units, if applicable?
3. What assumptions or conditions are required, and how would you verify your implementation?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Vector addition

Programming problem: Add two same-length vectors element by element; test zero vectors and mismatched lengths.

Questions you should be able to answer:

1. State the definition of vector addition and explain its intuitive or geometric meaning.
2. Work through one small numerical example by hand. What are the dimensions and units, if applicable?
3. What assumptions or conditions are required, and how would you verify your implementation?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Scalar multiplication

Programming problem: Multiply every vector component by a scalar and test zero, negative, and fractional scalars.

Questions you should be able to answer:

1. State the definition of scalar multiplication and explain its intuitive or geometric meaning.
2. Work through one small numerical example by hand. What are the dimensions and units, if applicable?
3. What assumptions or conditions are required, and how would you verify your implementation?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Dot products

Programming problem: Implement the dot product with a loop; compare parallel, perpendicular, and opposite vectors.

Questions you should be able to answer:

1. What does the dot product measure geometrically?
2. Why is a dot product zero for perpendicular vectors?
3. How does the result change when one vector is scaled?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Vector norms

Programming problem: Implement L1 and L2 norms; verify non-negativity and that the zero vector has norm zero.

Questions you should be able to answer:

1. State the definition of vector norms and explain its intuitive or geometric meaning.
2. Work through one small numerical example by hand. What are the dimensions and units, if applicable?
3. What assumptions or conditions are required, and how would you verify your implementation?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Distances

Programming problem: Implement Euclidean distance between points and use it to choose the nearest of several points.

Questions you should be able to answer:

1. State the definition of distances and explain its intuitive or geometric meaning.
2. Work through one small numerical example by hand. What are the dimensions and units, if applicable?
3. What assumptions or conditions are required, and how would you verify your implementation?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Matrices

Programming problem: Represent a matrix as nested lists; validate that all rows have equal length and print its shape.

Questions you should be able to answer:

1. State the definition of matrices and explain its intuitive or geometric meaning.

2. Work through one small numerical example by hand. What are the dimensions and units, if applicable?
3. What assumptions or conditions are required, and how would you verify your implementation?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Matrix multiplication

Programming problem: Implement matrix multiplication with loops and reject incompatible shapes; verify a 2x2 example by hand.

Questions you should be able to answer:

1. Why must inner dimensions match?
2. What does applying a matrix to a vector mean geometrically?
3. Why is matrix multiplication generally not commutative?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Linear transformations

Programming problem: Apply a 2x2 matrix to several 2D points and record how the matrix changes basis vectors.

Questions you should be able to answer:

1. State the definition of linear transformations and explain its intuitive or geometric meaning.
2. Work through one small numerical example by hand. What are the dimensions and units, if applicable?
3. What assumptions or conditions are required, and how would you verify your implementation?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Identity matrices

Programming problem: Construct an identity matrix of arbitrary size and test that multiplying by it preserves a vector.

Questions you should be able to answer:

1. State the definition of identity matrices and explain its intuitive or geometric meaning.
2. Work through one small numerical example by hand. What are the dimensions and units, if applicable?
3. What assumptions or conditions are required, and how would you verify your implementation?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Matrix inverses

Programming problem: Implement the inverse of a 2x2 matrix and detect singular matrices using the determinant.

Questions you should be able to answer:

1. State the definition of matrix inverses and explain its intuitive or geometric meaning.
2. Work through one small numerical example by hand. What are the dimensions and units, if applicable?
3. What assumptions or conditions are required, and how would you verify your implementation?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Matrix rank

Programming problem: For small matrices, implement row reduction and count pivot rows; compare dependent and independent rows.

Questions you should be able to answer:

1. What does rank tell you about independent rows or columns?
2. How does rank relate to the number of solutions of a linear system?
3. What numerical issue can make rank detection difficult?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Systems of linear equations

Programming problem: Solve a small 2-variable linear system by elimination and check each solution in the original equations.

Questions you should be able to answer:

1. State the definition of systems of linear equations and explain its intuitive or geometric meaning.
2. Work through one small numerical example by hand. What are the dimensions and units, if applicable?
3. What assumptions or conditions are required, and how would you verify your implementation?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Basis

Programming problem: Test whether candidate 2D vectors are linearly independent and can represent selected points.

Questions you should be able to answer:

1. State the definition of basis and explain its intuitive or geometric meaning.
2. Work through one small numerical example by hand. What are the dimensions and units, if applicable?
3. What assumptions or conditions are required, and how would you verify your implementation?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Span

Programming problem: Generate linear combinations of two vectors and visualize the set of points they can reach in 2D.

Questions you should be able to answer:

1. State the definition of span and explain its intuitive or geometric meaning.
2. Work through one small numerical example by hand. What are the dimensions and units, if applicable?
3. What assumptions or conditions are required, and how would you verify your implementation?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Eigenvalues and eigenvectors

Programming problem: For a simple 2x2 matrix with known eigenvectors, verify A v = lambda v numerically.

Questions you should be able to answer:

1. State the definition of eigenvalues and eigenvectors and explain its intuitive or geometric meaning.
2. Work through one small numerical example by hand. What are the dimensions and units, if applicable?
3. What assumptions or conditions are required, and how would you verify your implementation?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Phase 2C — Calculus

Limits

Programming problem: Approximate a function’s limit by evaluating points closer to a target from both sides.

Questions you should be able to answer:

1. State the definition of limits and explain its intuitive or geometric meaning.
2. Work through one small numerical example by hand. What are the dimensions and units, if applicable?
3. What assumptions or conditions are required, and how would you verify your implementation?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Continuity

Programming problem: Sample a piecewise function around its join and detect a mismatch between one-sided limits and function value.

Questions you should be able to answer:

1. State the definition of continuity and explain its intuitive or geometric meaning.
2. Work through one small numerical example by hand. What are the dimensions and units, if applicable?
3. What assumptions or conditions are required, and how would you verify your implementation?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Derivatives

Programming problem: Estimate a derivative with a centered finite difference and compare it with a known derivative.

Questions you should be able to answer:

1. State the definition of derivatives and explain its intuitive or geometric meaning.
2. Work through one small numerical example by hand. What are the dimensions and units, if applicable?
3. What assumptions or conditions are required, and how would you verify your implementation?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Slopes as rates of change

Programming problem: Calculate average rate of change over intervals of different sizes and compare with a local estimate.

Questions you should be able to answer:

1. State the definition of slopes as rates of change and explain its intuitive or geometric meaning.
2. Work through one small numerical example by hand. What are the dimensions and units, if applicable?
3. What assumptions or conditions are required, and how would you verify your implementation?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Partial derivatives

Programming problem: Approximate each partial derivative of f(x,y)=xx+3xy+yy by changing one variable at a time.

Questions you should be able to answer:

1. State the definition of partial derivatives and explain its intuitive or geometric meaning.
2. Work through one small numerical example by hand. What are the dimensions and units, if applicable?
3. What assumptions or conditions are required, and how would you verify your implementation?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Gradients

Programming problem: Calculate the gradient of a two-variable quadratic and take one small step downhill.

Questions you should be able to answer:

1. What does a gradient point toward for a scalar-valued function?
2. Why is the negative gradient used for local minimization?
3. How can finite differences check a gradient implementation?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Integrals

Programming problem: Approximate an area under a curve using left, right, and midpoint Riemann sums.

Questions you should be able to answer:

1. State the definition of integrals and explain its intuitive or geometric meaning.
2. Work through one small numerical example by hand. What are the dimensions and units, if applicable?
3. What assumptions or conditions are required, and how would you verify your implementation?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Taylor approximations

Programming problem: Approximate exp(x) with a truncated Taylor series and measure error as x and the term count change.

Questions you should be able to answer:

1. State the definition of taylor approximations and explain its intuitive or geometric meaning.
2. Work through one small numerical example by hand. What are the dimensions and units, if applicable?
3. What assumptions or conditions are required, and how would you verify your implementation?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Phase 2D — Probability and statistics

Events

Programming problem: Simulate coin tosses and estimate the frequency of a defined event as trial count grows.

Questions you should be able to answer:

1. What quantity or uncertainty does events describe?
2. Calculate or reason through one tiny example by hand and explain the result in plain language.
3. Which assumptions or sampling effects could make a computed result misleading?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Conditional probability

Programming problem: Simulate two events and calculate P(A|B) from counts; compare with the theoretical result.

Questions you should be able to answer:

1. What quantity or uncertainty does conditional probability describe?
2. Calculate or reason through one tiny example by hand and explain the result in plain language.
3. Which assumptions or sampling effects could make a computed result misleading?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Bayes’ theorem

Programming problem: Create a small diagnostic-test example and calculate a posterior probability from prior and likelihood values.

Questions you should be able to answer:

1. How do prior, likelihood, and evidence combine into a posterior?
2. Why can a rare event remain unlikely after a positive test?
3. Which assumptions in your numerical example matter?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Random variables

Programming problem: Simulate a discrete random variable from a probability table and verify observed frequencies.

Questions you should be able to answer:

1. What quantity or uncertainty does random variables describe?
2. Calculate or reason through one tiny example by hand and explain the result in plain language.
3. Which assumptions or sampling effects could make a computed result misleading?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Expectation

Programming problem: Compute the expected value from a discrete distribution and compare it with the average of many simulations.

Questions you should be able to answer:

1. What quantity or uncertainty does expectation describe?
2. Calculate or reason through one tiny example by hand and explain the result in plain language.
3. Which assumptions or sampling effects could make a computed result misleading?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Variance

Programming problem: Calculate variance by hand from a small list and compare population variance with sample variance.

Questions you should be able to answer:

1. What quantity or uncertainty does variance describe?
2. Calculate or reason through one tiny example by hand and explain the result in plain language.
3. Which assumptions or sampling effects could make a computed result misleading?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Covariance

Programming problem: Compute covariance for pairs of values that move together and in opposite directions.

Questions you should be able to answer:

1. What quantity or uncertainty does covariance describe?
2. Calculate or reason through one tiny example by hand and explain the result in plain language.
3. Which assumptions or sampling effects could make a computed result misleading?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Common probability distributions

Programming problem: Sample from a Bernoulli and a uniform distribution; compare histograms and empirical means with theory.

Questions you should be able to answer:

1. What quantity or uncertainty does common probability distributions describe?
2. Calculate or reason through one tiny example by hand and explain the result in plain language.
3. Which assumptions or sampling effects could make a computed result misleading?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Sampling

Programming problem: Implement greedy decoding and multinomial sampling from a short list of token probabilities.

Questions you should be able to answer:

1. What quantity or uncertainty does sampling describe?
2. Calculate or reason through one tiny example by hand and explain the result in plain language.
3. Which assumptions or sampling effects could make a computed result misleading?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Likelihood

Programming problem: Compute likelihoods of observed coin tosses for several candidate probabilities and compare which fits best.

Questions you should be able to answer:

1. What quantity or uncertainty does likelihood describe?
2. Calculate or reason through one tiny example by hand and explain the result in plain language.
3. Which assumptions or sampling effects could make a computed result misleading?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Entropy

Programming problem: Calculate Shannon entropy for a fair coin, a biased coin, and a certain outcome.

Questions you should be able to answer:

1. What does entropy measure about a probability distribution?
2. Why is entropy largest for a uniform distribution over a fixed number of outcomes?
3. What happens to entropy when one outcome becomes certain?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Cross-entropy

Programming problem: Compute cross-entropy for a target class under two predicted probability distributions.

Questions you should be able to answer:

1. What quantity or uncertainty does cross-entropy describe?
2. Calculate or reason through one tiny example by hand and explain the result in plain language.
3. Which assumptions or sampling effects could make a computed result misleading?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Phase 3 — Numerical computing

Floating-point arithmetic

Programming problem: Show that 0.1+0.2 may not equal exactly 0.3 in binary floating-point and test with a tolerance.

Questions you should be able to answer:

1. What does floating-point arithmetic mean for computation on a finite-precision machine?
2. What intermediate values or shapes determine the result?
3. Which edge case can produce an inaccurate or invalid result, and how can you detect it?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Numerical precision

Programming problem: Compare two close floating-point values using absolute and relative error.

Questions you should be able to answer:

1. What does numerical precision mean for computation on a finite-precision machine?
2. What intermediate values or shapes determine the result?
3. Which edge case can produce an inaccurate or invalid result, and how can you detect it?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Overflow

Programming problem: Use fixed-width integer or floating-point examples to demonstrate overflow behavior where supported; explain limits.

Questions you should be able to answer:

1. What does overflow mean for computation on a finite-precision machine?
2. What intermediate values or shapes determine the result?
3. Which edge case can produce an inaccurate or invalid result, and how can you detect it?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Underflow

Programming problem: Evaluate extremely small products and observe when values round toward zero.

Questions you should be able to answer:

1. What does underflow mean for computation on a finite-precision machine?

2. What intermediate values or shapes determine the result?
3. Which edge case can produce an inaccurate or invalid result, and how can you detect it?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Broadcasting

Programming problem: Implement a tiny broadcasting example for a vector plus a scalar and describe compatible shapes.

Questions you should be able to answer:

1. What does broadcasting mean for computation on a finite-precision machine?
2. What intermediate values or shapes determine the result?
3. Which edge case can produce an inaccurate or invalid result, and how can you detect it?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Vectorization

Programming problem: Compute the same element-wise transformation with loops and NumPy, then compare outputs and timing.

Questions you should be able to answer:

1. What does vectorization mean for computation on a finite-precision machine?
2. What intermediate values or shapes determine the result?
3. Which edge case can produce an inaccurate or invalid result, and how can you detect it?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Array shapes

Programming problem: Write shape-checking helpers for vectors and matrices and test mismatched dimensions.

Questions you should be able to answer:

1. What does array shapes mean for computation on a finite-precision machine?
2. What intermediate values or shapes determine the result?
3. Which edge case can produce an inaccurate or invalid result, and how can you detect it?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Tensor dimensions

Programming problem: Label batch, sequence, feature, and channel axes for small toy tensors; assert expected dimensions.

Questions you should be able to answer:

1. What does tensor dimensions mean for computation on a finite-precision machine?
2. What intermediate values or shapes determine the result?
3. Which edge case can produce an inaccurate or invalid result, and how can you detect it?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Numerical stability

Programming problem: Compute softmax for large logits, then stabilize it by subtracting the maximum before exponentiation.

Questions you should be able to answer:

1. Why can mathematically equivalent formulas have different numerical behavior?
2. Why do we subtract the maximum logit before softmax?
3. How would you test a function on extreme inputs?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Random number generation

Programming problem: Set a seed, reproduce a sequence, and show how changing the seed changes results.

Questions you should be able to answer:

1. What does random number generation mean for computation on a finite-precision machine?
2. What intermediate values or shapes determine the result?
3. Which edge case can produce an inaccurate or invalid result, and how can you detect it?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Matrix operations

Programming problem: Implement transpose, addition, matrix-vector multiplication, and shape errors using loops.

Questions you should be able to answer:

1. What does matrix operations mean for computation on a finite-precision machine?
2. What intermediate values or shapes determine the result?
3. Which edge case can produce an inaccurate or invalid result, and how can you detect it?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Finite differences

Programming problem: Compare forward, backward, and centered finite differences on a known smooth function.

Questions you should be able to answer:

1. What does finite differences mean for computation on a finite-precision machine?
2. What intermediate values or shapes determine the result?
3. Which edge case can produce an inaccurate or invalid result, and how can you detect it?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Numerical approximation

Programming problem: Approximate pi or an integral with increasing sample counts and graph the error.

Questions you should be able to answer:

1. What does numerical approximation mean for computation on a finite-precision machine?
2. What intermediate values or shapes determine the result?
3. Which edge case can produce an inaccurate or invalid result, and how can you detect it?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Phase 4A — Machine-learning foundations

Datasets

Programming problem: Create a tiny dataset object with length, indexing, and train/test splitting; verify no overlap.

Questions you should be able to answer:

1. What problem does datasets solve, and how does it produce a prediction or update?
2. Which inputs, intermediate values, objective, or assumptions does it depend on?
3. How would you test it on a tiny dataset, and what failure mode would you look for?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Features

Programming problem: Represent examples as feature rows and show how changing one feature changes a model prediction.

Questions you should be able to answer:

1. What problem does features solve, and how does it produce a prediction or update?
2. Which inputs, intermediate values, objective, or assumptions does it depend on?
3. How would you test it on a tiny dataset, and what failure mode would you look for?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Targets

Programming problem: Separate input features from target labels and write checks that catch misaligned rows.

Questions you should be able to answer:

1. What problem does targets solve, and how does it produce a prediction or update?
2. Which inputs, intermediate values, objective, or assumptions does it depend on?
3. How would you test it on a tiny dataset, and what failure mode would you look for?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Parameters

Programming problem: Create a model parameter dictionary and show how different values change predictions.

Questions you should be able to answer:

1. What problem does parameters solve, and how does it produce a prediction or update?
2. Which inputs, intermediate values, objective, or assumptions does it depend on?
3. How would you test it on a tiny dataset, and what failure mode would you look for?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Hypotheses

Programming problem: Implement two candidate prediction functions and compare their errors on the same data.

Questions you should be able to answer:

1. What problem does hypotheses solve, and how does it produce a prediction or update?
2. Which inputs, intermediate values, objective, or assumptions does it depend on?
3. How would you test it on a tiny dataset, and what failure mode would you look for?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Training

Programming problem: Write a loop that repeatedly predicts, measures loss, and updates model parameters.

Questions you should be able to answer:

1. What problem does training solve, and how does it produce a prediction or update?
2. Which inputs, intermediate values, objective, or assumptions does it depend on?
3. How would you test it on a tiny dataset, and what failure mode would you look for?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Inference

Programming problem: Freeze trained parameters and make predictions without updating them; verify parameters stay unchanged.

Questions you should be able to answer:

1. What problem does inference solve, and how does it produce a prediction or update?
2. Which inputs, intermediate values, objective, or assumptions does it depend on?
3. How would you test it on a tiny dataset, and what failure mode would you look for?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Loss functions

Programming problem: Implement squared error and absolute error for the same predictions and compare their penalties.

Questions you should be able to answer:

1. What problem does loss functions solve, and how does it produce a prediction or update?
2. Which inputs, intermediate values, objective, or assumptions does it depend on?
3. How would you test it on a tiny dataset, and what failure mode would you look for?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Optimization

Programming problem: Train a tiny language model while recording loss and confirming parameters change after optimizer steps.

Questions you should be able to answer:

1. What problem does optimization solve, and how does it produce a prediction or update?
2. Which inputs, intermediate values, objective, or assumptions does it depend on?
3. How would you test it on a tiny dataset, and what failure mode would you look for?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Gradient descent

Programming problem: Minimize f(w)=(w-3)^2 using the derivative; log w and loss at every step.

Questions you should be able to answer:

1. How do the gradient and learning rate determine the parameter update?
2. Why may a large learning rate diverge?
3. How can you tell whether a gradient sign is wrong?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Overfitting

Programming problem: Train a high-degree polynomial on noisy data and compare training error with held-out error.

Questions you should be able to answer:

1. Why can training error be low while test error is high?
2. How do model complexity and dataset size affect overfitting?
3. How do validation data and regularization help?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Underfitting

Programming problem: Fit a straight line to clearly curved data and diagnose the high error on both train and test sets.

Questions you should be able to answer:

1. What problem does underfitting solve, and how does it produce a prediction or update?
2. Which inputs, intermediate values, objective, or assumptions does it depend on?
3. How would you test it on a tiny dataset, and what failure mode would you look for?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Train-validation-test split

Programming problem: Split synthetic data reproducibly and ensure test data is not used to choose hyperparameters.

Questions you should be able to answer:

1. What problem does train-validation-test split solve, and how does it produce a prediction or update?
2. Which inputs, intermediate values, objective, or assumptions does it depend on?

3. How would you test it on a tiny dataset, and what failure mode would you look for?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Regularization

Programming problem: Add an L2 penalty to a regression objective and compare learned parameter magnitudes.

Questions you should be able to answer:

1. What problem does regularization solve, and how does it produce a prediction or update?
2. Which inputs, intermediate values, objective, or assumptions does it depend on?
3. How would you test it on a tiny dataset, and what failure mode would you look for?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Evaluation metrics

Programming problem: Calculate accuracy, precision, recall, F1, and mean squared error on tiny known predictions.

Questions you should be able to answer:

1. What problem does evaluation metrics solve, and how does it produce a prediction or update?
2. Which inputs, intermediate values, objective, or assumptions does it depend on?
3. How would you test it on a tiny dataset, and what failure mode would you look for?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Generalization

Programming problem: Compare train and test performance as model complexity changes on a generated dataset.

Questions you should be able to answer:

1. What problem does generalization solve, and how does it produce a prediction or update?
2. Which inputs, intermediate values, objective, or assumptions does it depend on?
3. How would you test it on a tiny dataset, and what failure mode would you look for?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Phase 4B — Classical machine-learning algorithms

Linear regression

Programming problem: Implement y_hat=w*x+b, calculate loss, derive gradients, and train w and b without scikit-learn.

Questions you should be able to answer:

1. How do weights and bias determine the predictions?
2. What objective is minimized by ordinary least squares?
3. What happens if features have very different scales or are collinear?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Mean squared error

Programming problem: Calculate mean squared error manually and verify how doubling a residual affects its squared contribution.

Questions you should be able to answer:

1. What problem does mean squared error solve, and how does it produce a prediction or update?
2. Which inputs, intermediate values, objective, or assumptions does it depend on?
3. How would you test it on a tiny dataset, and what failure mode would you look for?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Logistic regression

Programming problem: Implement a binary classifier with sigmoid and gradient descent on a tiny two-feature dataset.

Questions you should be able to answer:

1. Why does logistic regression produce a probability-like output?
2. How is its decision threshold used for classification?
3. Why is it still a linear model in feature space?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Sigmoid function

Programming problem: Implement sigmoid with a numerically stable form and inspect outputs for negative, zero, and large positive inputs.

Questions you should be able to answer:

1. What problem does sigmoid function solve, and how does it produce a prediction or update?
2. Which inputs, intermediate values, objective, or assumptions does it depend on?
3. How would you test it on a tiny dataset, and what failure mode would you look for?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Binary cross-entropy

Programming problem: Calculate binary cross-entropy for correct and incorrect confident predictions; safely handle probability bounds.

Questions you should be able to answer:

1. What problem does binary cross-entropy solve, and how does it produce a prediction or update?
2. Which inputs, intermediate values, objective, or assumptions does it depend on?
3. How would you test it on a tiny dataset, and what failure mode would you look for?

☐ Implemented - [ ] Tested - [ ] Explained from memory

K-nearest neighbors

Programming problem: Implement KNN using Euclidean distance, sorting distances, and majority voting without scikit-learn.

Questions you should be able to answer:

1. What problem does k-nearest neighbors solve, and how does it produce a prediction or update?
2. Which inputs, intermediate values, objective, or assumptions does it depend on?
3. How would you test it on a tiny dataset, and what failure mode would you look for?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Distance metrics

Programming problem: Compare Euclidean, Manhattan, and cosine-based similarity on a small set of vectors.

Questions you should be able to answer:

1. What problem does distance metrics solve, and how does it produce a prediction or update?
2. Which inputs, intermediate values, objective, or assumptions does it depend on?
3. How would you test it on a tiny dataset, and what failure mode would you look for?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Naive Bayes

Programming problem: Train a simple word-count Naive Bayes spam classifier from a handful of labeled text messages.

Questions you should be able to answer:

1. What problem does naive bayes solve, and how does it produce a prediction or update?

2. Which inputs, intermediate values, objective, or assumptions does it depend on?
3. How would you test it on a tiny dataset, and what failure mode would you look for?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Conditional independence

Programming problem: Build a small probability table and compare a joint probability with the product of marginals.

Questions you should be able to answer:

1. What problem does conditional independence solve, and how does it produce a prediction or update?
2. Which inputs, intermediate values, objective, or assumptions does it depend on?
3. How would you test it on a tiny dataset, and what failure mode would you look for?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Decision trees

Programming problem: Build a depth-limited decision tree by selecting a split and recursively partitioning a toy dataset.

Questions you should be able to answer:

1. What problem does decision trees solve, and how does it produce a prediction or update?
2. Which inputs, intermediate values, objective, or assumptions does it depend on?
3. How would you test it on a tiny dataset, and what failure mode would you look for?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Entropy in decision trees

Programming problem: Calculate entropy for pure and mixed class labels and explain how impurity changes.

Questions you should be able to answer:

1. What problem does entropy in decision trees solve, and how does it produce a prediction or update?
2. Which inputs, intermediate values, objective, or assumptions does it depend on?
3. How would you test it on a tiny dataset, and what failure mode would you look for?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Information gain

Programming problem: Evaluate candidate splits and select the one with the largest reduction in entropy.

Questions you should be able to answer:

1. What problem does information gain solve, and how does it produce a prediction or update?
2. Which inputs, intermediate values, objective, or assumptions does it depend on?
3. How would you test it on a tiny dataset, and what failure mode would you look for?

☐ Implemented - [ ] Tested - [ ] Explained from memory

K-means clustering

Programming problem: Implement assign-to-nearest-centroid and centroid-update steps; plot assignments for simple 2D points.

Questions you should be able to answer:

1. What problem does k-means clustering solve, and how does it produce a prediction or update?
2. Which inputs, intermediate values, objective, or assumptions does it depend on?
3. How would you test it on a tiny dataset, and what failure mode would you look for?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Centroids

Programming problem: Calculate cluster means by coordinate and show how one outlier moves a centroid.

Questions you should be able to answer:

1. What problem does centroids solve, and how does it produce a prediction or update?
2. Which inputs, intermediate values, objective, or assumptions does it depend on?
3. How would you test it on a tiny dataset, and what failure mode would you look for?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Principal component analysis (PCA)

Programming problem: Center a tiny 2D dataset, calculate covariance, and project onto a known principal direction.

Questions you should be able to answer:

1. What problem does principal component analysis (pca) solve, and how does it produce a prediction or update?
2. Which inputs, intermediate values, objective, or assumptions does it depend on?
3. How would you test it on a tiny dataset, and what failure mode would you look for?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Covariance matrix

Programming problem: Calculate a covariance matrix from two features and inspect the diagonal and off-diagonal values.

Questions you should be able to answer:

1. What problem does covariance matrix solve, and how does it produce a prediction or update?
2. Which inputs, intermediate values, objective, or assumptions does it depend on?
3. How would you test it on a tiny dataset, and what failure mode would you look for?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Support vector machines

Programming problem: For a tiny linearly separable dataset, visualize candidate separating lines and compare their margins.

Questions you should be able to answer:

1. What problem does support vector machines solve, and how does it produce a prediction or update?
2. Which inputs, intermediate values, objective, or assumptions does it depend on?
3. How would you test it on a tiny dataset, and what failure mode would you look for?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Margins and hinge loss

Programming problem: Implement hinge loss max(0, 1-y*f(x)) and test points inside and outside the margin.

Questions you should be able to answer:

1. What problem does margins and hinge loss solve, and how does it produce a prediction or update?
2. Which inputs, intermediate values, objective, or assumptions does it depend on?
3. How would you test it on a tiny dataset, and what failure mode would you look for?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Ensemble methods

Programming problem: Combine predictions from several simple classifiers and compare ensemble errors with individual errors.

Questions you should be able to answer:

1. What problem does ensemble methods solve, and how does it produce a prediction or update?
2. Which inputs, intermediate values, objective, or assumptions does it depend on?
3. How would you test it on a tiny dataset, and what failure mode would you look for?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Bagging

Programming problem: Train several decision stumps on bootstrap samples and aggregate their predictions.

Questions you should be able to answer:

1. What problem does bagging solve, and how does it produce a prediction or update?
2. Which inputs, intermediate values, objective, or assumptions does it depend on?
3. How would you test it on a tiny dataset, and what failure mode would you look for?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Random forests

Programming problem: Build multiple randomized trees on toy data and compare with a single tree; document randomness.

Questions you should be able to answer:

1. What problem does random forests solve, and how does it produce a prediction or update?
2. Which inputs, intermediate values, objective, or assumptions does it depend on?
3. How would you test it on a tiny dataset, and what failure mode would you look for?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Boosting

Programming problem: Implement a very small sequence of weak learners where each step focuses more on previous errors.

Questions you should be able to answer:

1. What problem does boosting solve, and how does it produce a prediction or update?
2. Which inputs, intermediate values, objective, or assumptions does it depend on?
3. How would you test it on a tiny dataset, and what failure mode would you look for?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Phase 5 — Neural-network foundations

Perceptron

Programming problem: Implement perceptron prediction and the mistake-driven weight update on linearly separable points.

Questions you should be able to answer:

1. Where does perceptron fit into the forward pass or training process?
2. Write the key equation and compute a small example by hand.
3. What happens to outputs, gradients, or learning if one input or parameter is changed?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Artificial neuron

Programming problem: Compute z=w dot x+b and apply an activation function for one neuron with hand-chosen values.

Questions you should be able to answer:

1. Where does artificial neuron fit into the forward pass or training process?
2. Write the key equation and compute a small example by hand.
3. What happens to outputs, gradients, or learning if one input or parameter is changed?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Artificial neural networks (ANNs)

Programming problem: Build a tiny feed-forward network from individual neurons and compare its capacity with a single perceptron on a toy problem.

Questions you should be able to answer:

1. How do layers of artificial neurons compose into a network?
2. Why can a multilayer network represent functions a single perceptron cannot?
3. What are the forward pass, loss, and backward pass in training?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Weights

Programming problem: Change one weight at a time in a tiny model and measure how the output changes.

Questions you should be able to answer:

1. Where does weights fit into the forward pass or training process?
2. Write the key equation and compute a small example by hand.
3. What happens to outputs, gradients, or learning if one input or parameter is changed?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Bias

Programming problem: Compare a linear model with and without bias and show how the decision boundary shifts.

Questions you should be able to answer:

1. Where does bias fit into the forward pass or training process?
2. Write the key equation and compute a small example by hand.
3. What happens to outputs, gradients, or learning if one input or parameter is changed?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Activation functions

Programming problem: Implement step, sigmoid, tanh, and ReLU; print and plot outputs over negative and positive inputs.

Questions you should be able to answer:

1. Where does activation functions fit into the forward pass or training process?
2. Write the key equation and compute a small example by hand.
3. What happens to outputs, gradients, or learning if one input or parameter is changed?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Linear layers

Programming problem: Implement $y=Wx+b$ for a batch of small vectors with loops and verify dimensions.

Questions you should be able to answer:

1. Where does linear layers fit into the forward pass or training process?
2. Write the key equation and compute a small example by hand.
3. What happens to outputs, gradients, or learning if one input or parameter is changed?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Multilayer perceptron (MLP)

Programming problem: Build a 2-layer network with a forward pass and train it on XOR using manual gradients.

Questions you should be able to answer:

1. Where does multilayer perceptron (mlp) fit into the forward pass or training process?
2. Write the key equation and compute a small example by hand.
3. What happens to outputs, gradients, or learning if one input or parameter is changed?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Forward pass

Programming problem: Print every intermediate activation for a tiny network and verify the output by hand.

Questions you should be able to answer:

1. Where does forward pass fit into the forward pass or training process?
2. Write the key equation and compute a small example by hand.
3. What happens to outputs, gradients, or learning if one input or parameter is changed?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Loss calculation

Programming problem: Calculate prediction loss for a batch, then compare per-example and averaged loss.

Questions you should be able to answer:

1. Where does loss calculation fit into the forward pass or training process?
2. Write the key equation and compute a small example by hand.
3. What happens to outputs, gradients, or learning if one input or parameter is changed?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Computational graph

Programming problem: Represent a scalar expression as nodes and calculate intermediate values in topological order.

Questions you should be able to answer:

1. Where does computational graph fit into the forward pass or training process?
2. Write the key equation and compute a small example by hand.
3. What happens to outputs, gradients, or learning if one input or parameter is changed?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Chain rule

Programming problem: For y=(2x+1)^2, calculate the derivative through intermediate steps and compare with numerical differences.

Questions you should be able to answer:

1. Where does chain rule fit into the forward pass or training process?
2. Write the key equation and compute a small example by hand.
3. What happens to outputs, gradients, or learning if one input or parameter is changed?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Backpropagation

Programming problem: Derive and implement gradients for a one-neuron model; compare against finite-difference gradients.

Questions you should be able to answer:

1. How does the chain rule pass error information backward?
2. What is the difference between a gradient and a parameter update?
3. Why compare analytical gradients with finite differences?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Gradient descent for neural networks

Programming problem: Train a tiny neural network and log loss, gradient norms, and parameter values over updates.

Questions you should be able to answer:

1. Where does gradient descent for neural networks fit into the forward pass or training process?

2. Write the key equation and compute a small example by hand.
3. What happens to outputs, gradients, or learning if one input or parameter is changed?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Stochastic gradient descent (SGD)

Programming problem: Compare full-batch and one-example-at-a-time updates on the same toy dataset.

Questions you should be able to answer:

1. Where does stochastic gradient descent (sgd) fit into the forward pass or training process?
2. Write the key equation and compute a small example by hand.
3. What happens to outputs, gradients, or learning if one input or parameter is changed?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Minibatches

Programming problem: Train using several batch sizes and compare gradient noise and loss curves.

Questions you should be able to answer:

1. Where does minibatches fit into the forward pass or training process?
2. Write the key equation and compute a small example by hand.
3. What happens to outputs, gradients, or learning if one input or parameter is changed?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Parameter initialization

Programming problem: Compare all-zero, small-random, and scaled-random initialization in a multilayer network.

Questions you should be able to answer:

1. Where does parameter initialization fit into the forward pass or training process?
2. Write the key equation and compute a small example by hand.
3. What happens to outputs, gradients, or learning if one input or parameter is changed?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Learning rate

Programming problem: Train the same model with too-small, moderate, and too-large learning rates; plot the losses.

Questions you should be able to answer:

1. Where does learning rate fit into the forward pass or training process?
2. Write the key equation and compute a small example by hand.
3. What happens to outputs, gradients, or learning if one input or parameter is changed?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Parameter updates

Programming problem: Apply one gradient update by hand and write an assertion that verifies the updated parameter.

Questions you should be able to answer:

1. Where does parameter updates fit into the forward pass or training process?
2. Write the key equation and compute a small example by hand.
3. What happens to outputs, gradients, or learning if one input or parameter is changed?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Phase 6A — Convolutional neural networks

Image tensors

Programming problem: Represent a grayscale image as a 2D list and an RGB image as height x width x channels; check indexing.

Questions you should be able to answer:

1. What role does image tensors play in converting an image tensor into features?
2. How do the input size, kernel, stride, padding, and channels determine its output?
3. How would you verify a tiny example manually and detect a shape/indexing bug?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Convolution

Programming problem: Implement a valid 2D convolution with nested loops on a small matrix and filter.

Questions you should be able to answer:

1. What operation does a kernel perform as it slides over an image?
2. How do stride and padding affect output size?
3. How is convolution different from a fully connected layer?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Kernels / filters

Programming problem: Apply edge-like and blur-like kernels to a tiny image and compare resulting feature maps.

Questions you should be able to answer:

1. What role does kernels / filters play in converting an image tensor into features?
2. How do the input size, kernel, stride, padding, and channels determine its output?
3. How would you verify a tiny example manually and detect a shape/indexing bug?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Stride

Programming problem: Extend convolution to stride 1 and stride 2; calculate output dimensions before executing.

Questions you should be able to answer:

1. What role does stride play in converting an image tensor into features?
2. How do the input size, kernel, stride, padding, and channels determine its output?
3. How would you verify a tiny example manually and detect a shape/indexing bug?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Padding

Programming problem: Implement zero padding and verify output dimensions with valid and same-style convolution.

Questions you should be able to answer:

1. What role does padding play in converting an image tensor into features?
2. How do the input size, kernel, stride, padding, and channels determine its output?
3. How would you verify a tiny example manually and detect a shape/indexing bug?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Feature maps

Programming problem: Print the output after each filter and compare which image patterns activate it.

Questions you should be able to answer:

1. What role does feature maps play in converting an image tensor into features?
2. How do the input size, kernel, stride, padding, and channels determine its output?
3. How would you verify a tiny example manually and detect a shape/indexing bug?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Pooling

Programming problem: Implement max-pooling and average-pooling over 2x2 regions with configurable stride.

Questions you should be able to answer:

1. What role does pooling play in converting an image tensor into features?
2. How do the input size, kernel, stride, padding, and channels determine its output?
3. How would you verify a tiny example manually and detect a shape/indexing bug?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Channels

Programming problem: Apply separate kernels to RGB channels and combine outputs; verify channel dimensions.

Questions you should be able to answer:

1. What role does channels play in converting an image tensor into features?
2. How do the input size, kernel, stride, padding, and channels determine its output?
3. How would you verify a tiny example manually and detect a shape/indexing bug?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Receptive fields

Programming problem: Calculate which input pixels can affect a selected output after stacked convolution layers.

Questions you should be able to answer:

1. What role does receptive fields play in converting an image tensor into features?
2. How do the input size, kernel, stride, padding, and channels determine its output?
3. How would you verify a tiny example manually and detect a shape/indexing bug?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Convolution gradients

Programming problem: Estimate the gradient of a tiny convolution loss with respect to one filter weight using finite differences.

Questions you should be able to answer:

1. What role does convolution gradients play in converting an image tensor into features?
2. How do the input size, kernel, stride, padding, and channels determine its output?
3. How would you verify a tiny example manually and detect a shape/indexing bug?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Phase 6B — Recurrent neural networks

Sequences

Programming problem: Represent a sequence of numbers and create input-target windows for next-step prediction.

Questions you should be able to answer:

1. How does sequences affect information as a sequence is processed over time?
2. What state and shared parameters are carried from one time step to the next?
3. What happens on a short sequence when you remove or alter this mechanism?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Hidden state

Programming problem: Implement a toy recurrent update h_t=tanh(Wx_t+Uh_(t-1)+b) and print the state over time.

Questions you should be able to answer:

1. How does hidden state affect information as a sequence is processed over time?
2. What state and shared parameters are carried from one time step to the next?
3. What happens on a short sequence when you remove or alter this mechanism?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Recurrent connections

Programming problem: Compare a recurrent model with the recurrent term removed and observe what information is lost.

Questions you should be able to answer:

1. How does recurrent connections affect information as a sequence is processed over time?
2. What state and shared parameters are carried from one time step to the next?
3. What happens on a short sequence when you remove or alter this mechanism?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Unrolling through time

Programming problem: Write out the repeated computation graph for a 3-step RNN and label shared parameters.

Questions you should be able to answer:

1. How does unrolling through time affect information as a sequence is processed over time?
2. What state and shared parameters are carried from one time step to the next?
3. What happens on a short sequence when you remove or alter this mechanism?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Backpropagation through time (BPTT)

Programming problem: Numerically estimate how the loss changes with a recurrent weight across a short sequence.

Questions you should be able to answer:

1. How does backpropagation through time (bptt) affect information as a sequence is processed over time?
2. What state and shared parameters are carried from one time step to the next?
3. What happens on a short sequence when you remove or alter this mechanism?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Vanishing gradients

Programming problem: Simulate repeated multiplication by a derivative smaller than one and plot gradient magnitude across steps.

Questions you should be able to answer:

1. Why can repeated derivatives smaller than one shrink gradients?
2. Which training behavior can result from very small gradients?
3. How do gating, initialization, and architecture influence the problem?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Exploding gradients

Programming problem: Simulate repeated multiplication by a factor greater than one and implement gradient-norm clipping.

Questions you should be able to answer:

1. How does exploding gradients affect information as a sequence is processed over time?
2. What state and shared parameters are carried from one time step to the next?
3. What happens on a short sequence when you remove or alter this mechanism?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Phase 6C — LSTM and GRU

LSTM cell

Programming problem: Implement a small LSTM step using sigmoid and tanh gates; print each gate and output state.

Questions you should be able to answer:

1. How does lstm cell affect information as a sequence is processed over time?
2. What state and shared parameters are carried from one time step to the next?
3. What happens on a short sequence when you remove or alter this mechanism?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Cell state

Programming problem: Trace cell state over several time steps and show how a gate can preserve or reduce it.

Questions you should be able to answer:

1. How does cell state affect information as a sequence is processed over time?
2. What state and shared parameters are carried from one time step to the next?
3. What happens on a short sequence when you remove or alter this mechanism?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Forget gate

Programming problem: Set forget-gate values near zero and one and compare retained information in the cell state.

Questions you should be able to answer:

1. How does forget gate affect information as a sequence is processed over time?
2. What state and shared parameters are carried from one time step to the next?
3. What happens on a short sequence when you remove or alter this mechanism?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Input gate

Programming problem: Vary an LSTM input gate and show how much candidate content is added to cell state.

Questions you should be able to answer:

1. How does input gate affect information as a sequence is processed over time?
2. What state and shared parameters are carried from one time step to the next?
3. What happens on a short sequence when you remove or alter this mechanism?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Output gate

Programming problem: Vary the output gate and compare visible hidden output while holding cell state fixed.

Questions you should be able to answer:

1. How does output gate affect information as a sequence is processed over time?
2. What state and shared parameters are carried from one time step to the next?
3. What happens on a short sequence when you remove or alter this mechanism?

☐ Implemented - [ ] Tested - [ ] Explained from memory

GRU update gate

Programming problem: Implement the GRU update equation and compare update-gate values near zero and one.

Questions you should be able to answer:

1. How does gru update gate affect information as a sequence is processed over time?
2. What state and shared parameters are carried from one time step to the next?
3. What happens on a short sequence when you remove or alter this mechanism?

☐ Implemented - [ ] Tested - [ ] Explained from memory

GRU reset gate

Programming problem: Vary the reset gate and observe how the previous state contributes to the candidate state.

Questions you should be able to answer:

1. How does gru reset gate affect information as a sequence is processed over time?
2. What state and shared parameters are carried from one time step to the next?
3. What happens on a short sequence when you remove or alter this mechanism?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Gated information flow

Programming problem: Create a tiny gated state update and plot how gate values control retained versus new information.

Questions you should be able to answer:

1. How does gated information flow affect information as a sequence is processed over time?
2. What state and shared parameters are carried from one time step to the next?
3. What happens on a short sequence when you remove or alter this mechanism?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Phase 7 — NLP, tokenization, and embeddings

Text normalization

Programming problem: Normalize whitespace and case in sample text while preserving a documented set of meaningful characters.

Questions you should be able to answer:

1. How does text normalization transform or represent text, and why is that step needed?
2. What information is preserved, changed, or lost in a small example?
3. How should unknown tokens, Unicode text, empty input, or encode/decode consistency be handled?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Unicode

Programming problem: Inspect code points for ASCII and non-ASCII text; demonstrate why visible characters and code points can differ.

Questions you should be able to answer:

1. How does unicode transform or represent text, and why is that step needed?
2. What information is preserved, changed, or lost in a small example?
3. How should unknown tokens, Unicode text, empty input, or encode/decode consistency be handled?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Bytes

Programming problem: Encode text to UTF-8 bytes and decode it back; test non-English text and invalid byte sequences.

Questions you should be able to answer:

1. How does bytes transform or represent text, and why is that step needed?
2. What information is preserved, changed, or lost in a small example?
3. How should unknown tokens, Unicode text, empty input, or encode/decode consistency be handled?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Character encoding

Programming problem: Round-trip several strings through UTF-8 and demonstrate a decoding error with the wrong encoding.

Questions you should be able to answer:

1. How does character encoding transform or represent text, and why is that step needed?
2. What information is preserved, changed, or lost in a small example?
3. How should unknown tokens, Unicode text, empty input, or encode/decode consistency be handled?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Vocabulary

Programming problem: Build a vocabulary from a corpus with frequency counts and a defined unknown-token strategy.

Questions you should be able to answer:

1. How does vocabulary transform or represent text, and why is that step needed?
2. What information is preserved, changed, or lost in a small example?
3. How should unknown tokens, Unicode text, empty input, or encode/decode consistency be handled?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Token IDs

Programming problem: Map tokens to integer IDs and back; check duplicate tokens and unknown-token behavior.

Questions you should be able to answer:

1. How does token ids transform or represent text, and why is that step needed?
2. What information is preserved, changed, or lost in a small example?
3. How should unknown tokens, Unicode text, empty input, or encode/decode consistency be handled?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Character tokenization

Programming problem: Encode text as characters and reconstruct the exact original string.

Questions you should be able to answer:

1. How does character tokenization transform or represent text, and why is that step needed?
2. What information is preserved, changed, or lost in a small example?
3. How should unknown tokens, Unicode text, empty input, or encode/decode consistency be handled?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Word tokenization

Programming problem: Implement a basic whitespace tokenizer and test punctuation, repeated spaces, and empty text.

Questions you should be able to answer:

1. How does word tokenization transform or represent text, and why is that step needed?
2. What information is preserved, changed, or lost in a small example?

3. How should unknown tokens, Unicode text, empty input, or encode/decode consistency be handled?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Subword tokenization

Programming problem: Split an unknown word into known subword pieces and calculate its token count.

Questions you should be able to answer:

1. How does subword tokenization transform or represent text, and why is that step needed?
2. What information is preserved, changed, or lost in a small example?
3. How should unknown tokens, Unicode text, empty input, or encode/decode consistency be handled?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Byte-pair encoding (BPE)

Programming problem: Train a small BPE tokenizer by repeatedly merging the most frequent adjacent pair; save the merge list.

Questions you should be able to answer:

1. How does byte-pair encoding (bpe) transform or represent text, and why is that step needed?
2. What information is preserved, changed, or lost in a small example?
3. How should unknown tokens, Unicode text, empty input, or encode/decode consistency be handled?

☐ Implemented - [ ] Tested - [ ] Explained from memory

BPE merge rules

Programming problem: Apply a fixed ordered list of merges to a token sequence and show how merge order affects the result.

Questions you should be able to answer:

1. How is the next pair chosen during BPE training?
2. Why must encoder and decoder use the same ordered merge rules?
3. How do merge choices affect vocabulary and sequence length?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Token-frequency analysis

Programming problem: Count token frequencies before and after BPE and report vocabulary size and average sequence length.

Questions you should be able to answer:

1. How does token-frequency analysis transform or represent text, and why is that step needed?
2. What information is preserved, changed, or lost in a small example?
3. How should unknown tokens, Unicode text, empty input, or encode/decode consistency be handled?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Embeddings

Programming problem: Implement an embedding lookup table as nested lists and retrieve vectors for a sequence of token IDs.

Questions you should be able to answer:

1. Why is a token ID not itself a meaningful semantic representation?
2. How are embedding values learned during training?
3. How could you inspect whether vector similarity captures a useful relationship?

☐ Implemented - [ ] Tested - [ ] Explained from memory

One-hot encoding

Programming problem: Convert class IDs into one-hot vectors and verify vector length and number of active entries.

Questions you should be able to answer:

1. How does one-hot encoding transform or represent text, and why is that step needed?
2. What information is preserved, changed, or lost in a small example?
3. How should unknown tokens, Unicode text, empty input, or encode/decode consistency be handled?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Cosine similarity

Programming problem: Implement cosine similarity and test identical, perpendicular, opposite, and zero vectors.

Questions you should be able to answer:

1. How does cosine similarity transform or represent text, and why is that step needed?
2. What information is preserved, changed, or lost in a small example?
3. How should unknown tokens, Unicode text, empty input, or encode/decode consistency be handled?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Semantic representation

Programming problem: Use a tiny hand-created vector table to rank similar terms; state clearly that the vectors are illustrative, not learned semantics.

Questions you should be able to answer:

1. How does semantic representation transform or represent text, and why is that step needed?
2. What information is preserved, changed, or lost in a small example?
3. How should unknown tokens, Unicode text, empty input, or encode/decode consistency be handled?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Positional information

Programming problem: Add simple positional vectors to token embeddings and demonstrate that swapping tokens changes the combined representations.

Questions you should be able to answer:

1. How does positional information transform or represent text, and why is that step needed?
2. What information is preserved, changed, or lost in a small example?
3. How should unknown tokens, Unicode text, empty input, or encode/decode consistency be handled?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Phase 8A — Transformer inputs and representations

Token embeddings

Programming problem: Look up the embedding vector for each token in a short sentence and inspect the resulting matrix shape.

Questions you should be able to answer:

1. What role does token embeddings play in processing token representations or predicting the next token?
2. Work through a tiny example and state the shapes of every input and output.
3. What breaks if the dimensions, masking, normalization, or training target is handled incorrectly?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Learned parameters

Programming problem: Initialize an embedding table with values, update one vector, and demonstrate that the parameters have changed.

Questions you should be able to answer:

1. What role does learned parameters play in processing token representations or predicting the next token?
2. Work through a tiny example and state the shapes of every input and output.

3. What breaks if the dimensions, masking, normalization, or training target is handled incorrectly?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Positional embeddings

Programming problem: Create learned or fixed positional vectors and add them to token embeddings position by position.

Questions you should be able to answer:

1. What role does positional embeddings play in processing token representations or predicting the next token?
2. Work through a tiny example and state the shapes of every input and output.
3. What breaks if the dimensions, masking, normalization, or training target is handled incorrectly?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Sequence length

Programming problem: Build input sequences of different lengths and show how padding or truncation changes the batch shape.

Questions you should be able to answer:

1. What role does sequence length play in processing token representations or predicting the next token?
2. Work through a tiny example and state the shapes of every input and output.
3. What breaks if the dimensions, masking, normalization, or training target is handled incorrectly?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Embedding dimension

Programming problem: Change a toy embedding dimension and track the shape and number of parameters in the embedding table.

Questions you should be able to answer:

1. What role does embedding dimension play in processing token representations or predicting the next token?
2. Work through a tiny example and state the shapes of every input and output.
3. What breaks if the dimensions, masking, normalization, or training target is handled incorrectly?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Tensor shape

Programming problem: Write shape assertions at every stage of a toy transformer forward pass.

Questions you should be able to answer:

1. What role does tensor shape play in processing token representations or predicting the next token?
2. Work through a tiny example and state the shapes of every input and output.
3. What breaks if the dimensions, masking, normalization, or training target is handled incorrectly?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Input representation

Programming problem: Transform a sentence into token IDs, embeddings, and positional representations, recording each intermediate object.

Questions you should be able to answer:

1. What role does input representation play in processing token representations or predicting the next token?
2. Work through a tiny example and state the shapes of every input and output.
3. What breaks if the dimensions, masking, normalization, or training target is handled incorrectly?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Phase 8B — Self-attention

Queries

Programming problem: Compute a query vector from a tiny input embedding using a small manually specified projection matrix.

Questions you should be able to answer:

1. What role does queries play in processing token representations or predicting the next token?
2. Work through a tiny example and state the shapes of every input and output.
3. What breaks if the dimensions, masking, normalization, or training target is handled incorrectly?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Keys

Programming problem: Compute key vectors for tokens and explain how they are used to score relevance to a query.

Questions you should be able to answer:

1. What role does keys play in processing token representations or predicting the next token?
2. Work through a tiny example and state the shapes of every input and output.
3. What breaks if the dimensions, masking, normalization, or training target is handled incorrectly?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Values

Programming problem: Compute value vectors and use attention weights to form a weighted output.

Questions you should be able to answer:

1. What role does values play in processing token representations or predicting the next token?
2. Work through a tiny example and state the shapes of every input and output.
3. What breaks if the dimensions, masking, normalization, or training target is handled incorrectly?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Dot-product attention scores

Programming problem: Calculate all query-key dot products for three small vectors and display the score matrix.

Questions you should be able to answer:

1. What role does dot-product attention scores play in processing token representations or predicting the next token?
2. Work through a tiny example and state the shapes of every input and output.
3. What breaks if the dimensions, masking, normalization, or training target is handled incorrectly?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Attention-score scaling

Programming problem: Divide dot-product scores by the square root of key dimension and compare softmax sharpness.

Questions you should be able to answer:

1. What role does attention-score scaling play in processing token representations or predicting the next token?
2. Work through a tiny example and state the shapes of every input and output.
3. What breaks if the dimensions, masking, normalization, or training target is handled incorrectly?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Softmax

Programming problem: Implement numerically stable softmax with loops; verify probabilities sum to one.

Questions you should be able to answer:

1. Why do softmax outputs sum to one?
2. How does increasing one logit affect the distribution?
3. Why is subtracting the maximum useful for numerical stability?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Weighted sums

Programming problem: Multiply a list of vectors by weights and sum them; test one-hot weights and equal weights.

Questions you should be able to answer:

1. What role does weighted sums play in processing token representations or predicting the next token?
2. Work through a tiny example and state the shapes of every input and output.
3. What breaks if the dimensions, masking, normalization, or training target is handled incorrectly?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Causal masks

Programming problem: Build a lower-triangular mask and verify that each token cannot attend to future positions.

Questions you should be able to answer:

1. Why must a decoder-only language model prevent access to future tokens?
2. What mask entries are allowed for position i?
3. How could a masking bug leak target information?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Attention matrices

Programming problem: Print and inspect an attention matrix; assert row sums equal one after masking and softmax.

Questions you should be able to answer:

1. What role does attention matrices play in processing token representations or predicting the next token?
2. Work through a tiny example and state the shapes of every input and output.
3. What breaks if the dimensions, masking, normalization, or training target is handled incorrectly?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Phase 8C — Transformer blocks

Multi-head attention

Programming problem: Implement two tiny attention heads with separate projections and concatenate their outputs.

Questions you should be able to answer:

1. What role does multi-head attention play in processing token representations or predicting the next token?
2. Work through a tiny example and state the shapes of every input and output.
3. What breaks if the dimensions, masking, normalization, or training target is handled incorrectly?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Linear projections

Programming problem: Implement matrix multiplication for Q, K, and V projections and verify output dimensions.

Questions you should be able to answer:

1. What role does linear projections play in processing token representations or predicting the next token?

2. Work through a tiny example and state the shapes of every input and output.
3. What breaks if the dimensions, masking, normalization, or training target is handled incorrectly?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Feed-forward networks

Programming problem: Implement two linear layers with a nonlinearity between them and apply the same network independently to each token.

Questions you should be able to answer:

1. What role does feed-forward networks play in processing token representations or predicting the next token?
2. Work through a tiny example and state the shapes of every input and output.
3. What breaks if the dimensions, masking, normalization, or training target is handled incorrectly?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Residual connections

Programming problem: Add a block input to its transformed output and compare gradient paths in a small computational graph.

Questions you should be able to answer:

1. What role does residual connections play in processing token representations or predicting the next token?
2. Work through a tiny example and state the shapes of every input and output.
3. What breaks if the dimensions, masking, normalization, or training target is handled incorrectly?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Layer normalization

Programming problem: Calculate mean and variance across a feature vector and normalize it; compare with normalization across the wrong axis.

Questions you should be able to answer:

1. Which axes are normalized in layer normalization?
2. How is it different from batch normalization?
3. Why does normalizing across the wrong axis change the meaning?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Dropout

Programming problem: Implement dropout with a fixed seed in training mode and compare with inference mode.

Questions you should be able to answer:

1. What role does dropout play in processing token representations or predicting the next token?
2. Work through a tiny example and state the shapes of every input and output.
3. What breaks if the dimensions, masking, normalization, or training target is handled incorrectly?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Parameter sharing

Programming problem: Use one projection matrix at every sequence position and verify the same object/values are reused.

Questions you should be able to answer:

1. What role does parameter sharing play in processing token representations or predicting the next token?
2. Work through a tiny example and state the shapes of every input and output.
3. What breaks if the dimensions, masking, normalization, or training target is handled incorrectly?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Transformer tensor shapes

Programming problem: Write down and assert batch, sequence, head, and feature dimensions through a tiny attention operation.

Questions you should be able to answer:

1. What role does transformer tensor shapes play in processing token representations or predicting the next token?
2. Work through a tiny example and state the shapes of every input and output.
3. What breaks if the dimensions, masking, normalization, or training target is handled incorrectly?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Phase 8D — Training and generation

Vocabulary logits

Programming problem: Compute vocabulary logits from a hidden vector and a small output projection matrix.

Questions you should be able to answer:

1. What role does vocabulary logits play in processing token representations or predicting the next token?
2. Work through a tiny example and state the shapes of every input and output.
3. What breaks if the dimensions, masking, normalization, or training target is handled incorrectly?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Softmax for token probabilities

Programming problem: Convert logits into probabilities and test how a large logit changes the predicted distribution.

Questions you should be able to answer:

1. What role does softmax for token probabilities play in processing token representations or predicting the next token?
2. Work through a tiny example and state the shapes of every input and output.
3. What breaks if the dimensions, masking, normalization, or training target is handled incorrectly?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Next-token prediction

Programming problem: Create input-target token pairs shifted by one position and train a simple frequency-based baseline.

Questions you should be able to answer:

1. How are input tokens and target tokens shifted?
2. Why can next-token prediction train a model without human labels for every token?
3. How is token-level loss aggregated across a sequence?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Cross-entropy loss for language models

Programming problem: Compute the negative log probability of each target token and average over valid positions.

Questions you should be able to answer:

1. What role does cross-entropy loss for language models play in processing token representations or predicting the next token?
2. Work through a tiny example and state the shapes of every input and output.
3. What breaks if the dimensions, masking, normalization, or training target is handled incorrectly?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Causal language modeling

Programming problem: Build shifted input/target sequences and verify that predictions are trained against the next token, not the same token.

Questions you should be able to answer:

1. What role does causal language modeling play in processing token representations or predicting the next token?
2. Work through a tiny example and state the shapes of every input and output.
3. What breaks if the dimensions, masking, normalization, or training target is handled incorrectly?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Backpropagation in a transformer

Programming problem: For a tiny projection or attention parameter, compare analytical or autodiff gradients with finite differences.

Questions you should be able to answer:

1. What role does backpropagation in a transformer play in processing token representations or predicting the next token?
2. Work through a tiny example and state the shapes of every input and output.
3. What breaks if the dimensions, masking, normalization, or training target is handled incorrectly?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Optimization

Programming problem: Train a tiny language model while recording loss and confirming parameters change after optimizer steps.

Questions you should be able to answer:

1. What role does optimization play in processing token representations or predicting the next token?
2. Work through a tiny example and state the shapes of every input and output.
3. What breaks if the dimensions, masking, normalization, or training target is handled incorrectly?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Sampling

Programming problem: Implement greedy decoding and multinomial sampling from a short list of token probabilities.

Questions you should be able to answer:

1. What role does sampling play in processing token representations or predicting the next token?
2. Work through a tiny example and state the shapes of every input and output.
3. What breaks if the dimensions, masking, normalization, or training target is handled incorrectly?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Temperature

Programming problem: Apply temperature scaling to logits and compare distributions at low, unit, and high temperatures.

Questions you should be able to answer:

1. What role does temperature play in processing token representations or predicting the next token?
2. Work through a tiny example and state the shapes of every input and output.
3. What breaks if the dimensions, masking, normalization, or training target is handled incorrectly?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Context window

Programming problem: Truncate or window a long token stream and count how many training examples can be formed.

Questions you should be able to answer:

1. What limits the amount of prior text available to a model at one time?
2. What happens when the prompt exceeds the context window?
3. How do longer sequences affect attention computation and memory?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Inference and autoregressive generation

Programming problem: Generate one token at a time, append it to context, and stop at an end token or length limit.

Questions you should be able to answer:

1. What role does inference and autoregressive generation play in processing token representations or predicting the next token?
2. Work through a tiny example and state the shapes of every input and output.
3. What breaks if the dimensions, masking, normalization, or training target is handled incorrectly?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Phase 9 — Software engineering and project management

Git version control

Programming problem: Create a repository, make two commits, inspect the diff, and restore a file to a previous version.

Questions you should be able to answer:

1. What does Git store that makes history useful?
2. How do working tree, staging area, and commit differ?
3. How would you safely experiment without losing a working version?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Commits

Programming problem: Make a commit for one coherent change and explain what the commit message communicates.

Questions you should be able to answer:

1. What responsibility does commits have in a maintainable software project?
2. What interface, artifact, or observable result should it produce?
3. How would you test it, and what trade-off or failure case should be documented?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Branches

Programming problem: Create a feature branch, change a file, and merge it back after reviewing the diff.

Questions you should be able to answer:

1. What responsibility does branches have in a maintainable software project?
2. What interface, artifact, or observable result should it produce?
3. How would you test it, and what trade-off or failure case should be documented?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Merges

Programming problem: Create two branch changes and resolve a deliberate text conflict without losing either intended change.

Questions you should be able to answer:

1. What responsibility does merges have in a maintainable software project?
2. What interface, artifact, or observable result should it produce?
3. How would you test it, and what trade-off or failure case should be documented?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Dependency management

Programming problem: Record project dependencies and demonstrate that a clean environment can install them from the manifest.

Questions you should be able to answer:

1. What responsibility does dependency management have in a maintainable software project?
2. What interface, artifact, or observable result should it produce?
3. How would you test it, and what trade-off or failure case should be documented?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Virtual environments

Programming problem: Create an isolated environment, install one package, and verify it is not installed in the global interpreter.

Questions you should be able to answer:

1. What responsibility does virtual environments have in a maintainable software project?
2. What interface, artifact, or observable result should it produce?
3. How would you test it, and what trade-off or failure case should be documented?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Unit testing

Programming problem: Test a pure function independently using expected results and boundary cases.

Questions you should be able to answer:

1. What responsibility does unit testing have in a maintainable software project?
2. What interface, artifact, or observable result should it produce?
3. How would you test it, and what trade-off or failure case should be documented?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Integration testing

Programming problem: Test the path from data loading through prediction and metric calculation with a tiny fixture dataset.

Questions you should be able to answer:

1. What responsibility does integration testing have in a maintainable software project?
2. What interface, artifact, or observable result should it produce?
3. How would you test it, and what trade-off or failure case should be documented?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Type hints in projects

Programming problem: Add type hints to public interfaces and run a static type checker if available; document what hints cannot prove.

Questions you should be able to answer:

1. What responsibility does type hints in projects have in a maintainable software project?
2. What interface, artifact, or observable result should it produce?
3. How would you test it, and what trade-off or failure case should be documented?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Logging

Programming problem: Replace debugging print statements with logging levels and capture a failure with context.

Questions you should be able to answer:

1. What responsibility does logging have in a maintainable software project?
2. What interface, artifact, or observable result should it produce?
3. How would you test it, and what trade-off or failure case should be documented?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Configuration files

Programming problem: Load model settings from JSON or TOML, validate required fields, and reject unknown values where appropriate.

Questions you should be able to answer:

1. What responsibility does configuration files have in a maintainable software project?
2. What interface, artifact, or observable result should it produce?
3. How would you test it, and what trade-off or failure case should be documented?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Package structure

Programming problem: Turn a script into an installable package with clear modules and a simple import smoke test.

Questions you should be able to answer:

1. What responsibility does package structure have in a maintainable software project?
2. What interface, artifact, or observable result should it produce?
3. How would you test it, and what trade-off or failure case should be documented?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Documentation

Programming problem: Write a README with installation, usage example, expected output, limitations and test instructions.

Questions you should be able to answer:

1. What responsibility does documentation have in a maintainable software project?
2. What interface, artifact, or observable result should it produce?
3. How would you test it, and what trade-off or failure case should be documented?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Requirements and acceptance criteria

Programming problem: Write testable acceptance criteria for a small model-training feature and implement tests for each criterion.

Questions you should be able to answer:

1. What responsibility does requirements and acceptance criteria have in a maintainable software project?
2. What interface, artifact, or observable result should it produce?
3. How would you test it, and what trade-off or failure case should be documented?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Issue tracking

Programming problem: Create issues for a feature, break it into tasks, mark dependencies, and close only after tests pass.

Questions you should be able to answer:

1. What responsibility does issue tracking have in a maintainable software project?

2. What interface, artifact, or observable result should it produce?
3. How would you test it, and what trade-off or failure case should be documented?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Milestones

Programming problem: Plan a milestone with deliverables and completion tests; split work into tasks small enough to verify.

Questions you should be able to answer:

1. What responsibility does milestones have in a maintainable software project?
2. What interface, artifact, or observable result should it produce?
3. How would you test it, and what trade-off or failure case should be documented?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Code reviews

Programming problem: Review a deliberately flawed function for correctness, readability, tests and edge cases; write actionable review comments.

Questions you should be able to answer:

1. What responsibility does code reviews have in a maintainable software project?
2. What interface, artifact, or observable result should it produce?
3. How would you test it, and what trade-off or failure case should be documented?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Reproducible experiments

Programming problem: Run an experiment twice with the same seed and configuration; compare recorded outputs and document nondeterminism.

Questions you should be able to answer:

1. Which settings and artifacts must be recorded to reproduce a run?
2. Why might the same seed not guarantee bitwise-identical GPU results?
3. How can you distinguish a real improvement from random variation?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Release management

Programming problem: Create a versioned release checklist including tests, changelog, dependency versions and rollback notes.

Questions you should be able to answer:

1. What responsibility does release management have in a maintainable software project?
2. What interface, artifact, or observable result should it produce?
3. How would you test it, and what trade-off or failure case should be documented?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Scope definition

Programming problem: Write what a project will and will not do, plus measurable success criteria, before coding.

Questions you should be able to answer:

1. What responsibility does scope definition have in a maintainable software project?
2. What interface, artifact, or observable result should it produce?
3. How would you test it, and what trade-off or failure case should be documented?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Task decomposition

Programming problem: Break “train a classifier” into data, model, loss, optimizer, evaluation, tests and documentation tasks.

Questions you should be able to answer:

1. What responsibility does task decomposition have in a maintainable software project?
2. What interface, artifact, or observable result should it produce?
3. How would you test it, and what trade-off or failure case should be documented?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Module interfaces

Programming problem: Define input/output contracts for a dataset, model, optimizer and trainer; implement a small integration test.

Questions you should be able to answer:

1. What responsibility does module interfaces have in a maintainable software project?
2. What interface, artifact, or observable result should it produce?
3. How would you test it, and what trade-off or failure case should be documented?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Edge-case testing

Programming problem: Test empty input, one-item input, invalid shapes, NaN/Inf values, and boundary conditions for a numerical function.

Questions you should be able to answer:

1. What responsibility does edge-case testing have in a maintainable software project?
2. What interface, artifact, or observable result should it produce?
3. How would you test it, and what trade-off or failure case should be documented?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Experiment tracking

Programming problem: Save configuration, metrics, model parameters and timestamps for multiple runs and create a comparison report.

Questions you should be able to answer:

1. What responsibility does experiment tracking have in a maintainable software project?
2. What interface, artifact, or observable result should it produce?
3. How would you test it, and what trade-off or failure case should be documented?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Design decisions

Programming problem: Write a short decision record comparing two implementation choices, their trade-offs, and the reason for the selection.

Questions you should be able to answer:

1. What responsibility does design decisions have in a maintainable software project?
2. What interface, artifact, or observable result should it produce?
3. How would you test it, and what trade-off or failure case should be documented?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Phase 10 — Integrated ML laboratory project

Mathematics module

Programming problem: Create a module for vector and matrix operations with docstrings, tests and no dependency on NumPy.

Questions you should be able to answer:

1. What is mathematics module and what problem does it solve in the complete learning system?
2. What is the smallest useful implementation and how can you verify it independently?
3. How does it interact with neighboring modules, and what test would catch a regression?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Automatic differentiation engine

Programming problem: Represent a scalar expression as a computation graph, store local derivatives, and propagate gradients backward through the graph. Compare with finite differences.

Questions you should be able to answer:

1. How does automatic differentiation differ from symbolic differentiation and finite differences?
2. How are local derivatives combined through the chain rule on a computation graph?
3. How would you detect an incorrect gradient implementation?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Dataset loader

Programming problem: Load a small CSV dataset, validate columns and types, and report malformed rows clearly.

Questions you should be able to answer:

1. What is dataset loader and what problem does it solve in the complete learning system?
2. What is the smallest useful implementation and how can you verify it independently?
3. How does it interact with neighboring modules, and what test would catch a regression?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Preprocessing pipeline

Programming problem: Implement sequential preprocessing steps and test that training statistics are not computed from test data.

Questions you should be able to answer:

1. What is preprocessing pipeline and what problem does it solve in the complete learning system?
2. What is the smallest useful implementation and how can you verify it independently?
3. How does it interact with neighboring modules, and what test would catch a regression?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Model interface

Programming problem: Define a common predict() method and implement two models that satisfy the same interface.

Questions you should be able to answer:

1. What is model interface and what problem does it solve in the complete learning system?
2. What is the smallest useful implementation and how can you verify it independently?
3. How does it interact with neighboring modules, and what test would catch a regression?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Optimizer interface

Programming problem: Define a step() interface and implement a basic gradient-descent optimizer that updates a parameter dictionary.

Questions you should be able to answer:

1. What is optimizer interface and what problem does it solve in the complete learning system?
2. What is the smallest useful implementation and how can you verify it independently?
3. How does it interact with neighboring modules, and what test would catch a regression?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Training loop

Programming problem: Connect dataset batches, forward pass, loss, gradients, optimizer steps and logging in a clear loop.

Questions you should be able to answer:

1. What is training loop and what problem does it solve in the complete learning system?
2. What is the smallest useful implementation and how can you verify it independently?
3. How does it interact with neighboring modules, and what test would catch a regression?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Validation loop

Programming problem: Evaluate on held-out data without updating parameters; add an assertion that parameters do not change.

Questions you should be able to answer:

1. What is validation loop and what problem does it solve in the complete learning system?
2. What is the smallest useful implementation and how can you verify it independently?
3. How does it interact with neighboring modules, and what test would catch a regression?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Checkpointing

Programming problem: Save and reload model parameters and configuration; confirm predictions match after reload.

Questions you should be able to answer:

1. What is checkpointing and what problem does it solve in the complete learning system?
2. What is the smallest useful implementation and how can you verify it independently?
3. How does it interact with neighboring modules, and what test would catch a regression?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Metrics reporting

Programming problem: Generate a human-readable report with metric names, values, dataset split and run configuration.

Questions you should be able to answer:

1. What is metrics reporting and what problem does it solve in the complete learning system?
2. What is the smallest useful implementation and how can you verify it independently?
3. How does it interact with neighboring modules, and what test would catch a regression?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Experiment configuration

Programming problem: Store model, optimizer, data and seed settings in a configuration file and validate it before training.

Questions you should be able to answer:

1. What is experiment configuration and what problem does it solve in the complete learning system?
2. What is the smallest useful implementation and how can you verify it independently?
3. How does it interact with neighboring modules, and what test would catch a regression?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Reproducible random seeds

Programming problem: Set all relevant random seeds in a toy experiment and document which operations may still be nondeterministic.

Questions you should be able to answer:

1. What is reproducible random seeds and what problem does it solve in the complete learning system?

2. What is the smallest useful implementation and how can you verify it independently?
3. How does it interact with neighboring modules, and what test would catch a regression?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Gradient checking

Programming problem: Compare backpropagation gradients with centered finite differences for several parameters.

Questions you should be able to answer:

1. Why compare numerical and analytical gradients?
2. Why are centered differences often more accurate than forward differences?
3. Why should finite-difference checks use a tiny model and suitable step size?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Regression tests

Programming problem: Save known predictions or metrics and detect unintended changes after refactoring.

Questions you should be able to answer:

1. What is regression tests and what problem does it solve in the complete learning system?
2. What is the smallest useful implementation and how can you verify it independently?
3. How does it interact with neighboring modules, and what test would catch a regression?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Command-line interface

Programming problem: Build a CLI with train, evaluate and predict commands; validate arguments and return useful exit codes.

Questions you should be able to answer:

1. What is command-line interface and what problem does it solve in the complete learning system?
2. What is the smallest useful implementation and how can you verify it independently?
3. How does it interact with neighboring modules, and what test would catch a regression?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Model comparison

Programming problem: Run two models with identical data splits and compare metrics, runtime and model complexity.

Questions you should be able to answer:

1. What is model comparison and what problem does it solve in the complete learning system?
2. What is the smallest useful implementation and how can you verify it independently?
3. How does it interact with neighboring modules, and what test would catch a regression?

☐ Implemented - [ ] Tested - [ ] Explained from memory

Technical README

Programming problem: Document the project purpose, setup, architecture, commands, tests, results, limitations and next steps.

Questions you should be able to answer:

1. What is technical readme and what problem does it solve in the complete learning system?
2. What is the smallest useful implementation and how can you verify it independently?
3. How does it interact with neighboring modules, and what test would catch a regression?

☐ Implemented - [ ] Tested - [ ] Explained from memory